

EECE 5155: Wireless Sensor Networks and IoT Systems

Final Deliverable – Pipeline + ML/AI

Smart Watering and Humidity Control for Indoor Plants

Student Name: [Timothy Davis]

NUID: [002396048]

<https://github.com/davis-timothy/Smart-Watering-and-Humidity-Control-for-Indoor-Plants>

Date: [4/23/2026]

Version: 3.2

Part A: Data Pipeline and Cloud Integration (15 points)

1 System Architecture and Node-RED Flow

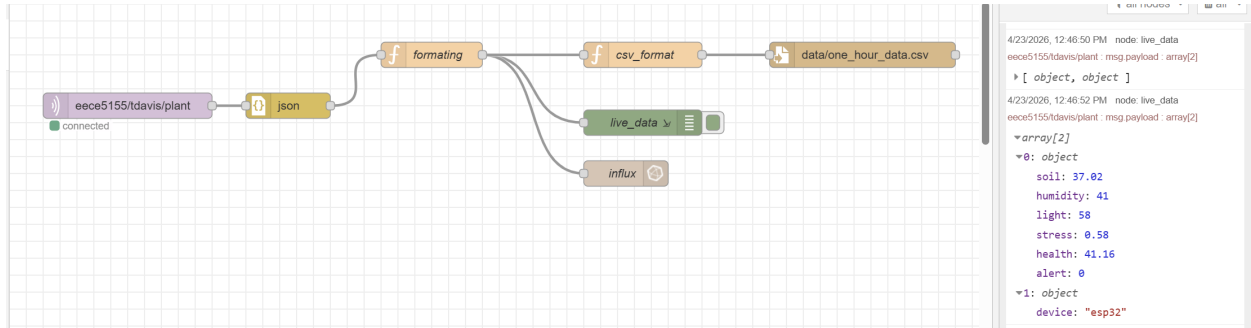
E1 Reference: Commit hash 0d0eb4975582a8102e05b43bbbed7188f2ae86125 — Changes since E1: Migrated MQTT broker configuration from Mosquitto to HiveMQ

Data Flow:

1. **Wokwi → MQTT:** The Wokwi ESP32 simulation generates plant monitoring data including soil moisture, humidity, light level, stress index, health score, and alert state. Readings are packaged in JSON format and published approximately every 2 seconds to the MQTT topic `eece5155/tdavis/plant` using the configured public MQTT broker.
2. **MQTT → Node-RED:** Node-RED subscribes to the topic using an MQTT input node labeled `smart_plant`. Incoming payloads pass through a JSON node, which converts the raw MQTT string payload into a structured message object.
3. **Node-RED Processing:** A function node labeled `formatting` extracts the relevant sensor values and converts them into an InfluxDB-compatible structure containing numeric fields and device tags. This step standardizes the telemetry for storage and visualization.
4. **Node-RED → CSV:** A parallel branch formats the incoming readings as comma-separated values and appends each sample to `data/one_hour_data.csv`. This creates a persistent dataset for offline analysis, descriptive statistics, and reproducibility.
5. **Node-RED → InfluxDB → Grafana:** The processed payload is simultaneously written to InfluxDB bucket `plant_data` under measurement `plant_metrics`. Grafana queries this database using Flux and displays live dashboards including soil moisture trends, humidity status, and plant health metrics. A debug branch is also maintained for real-time verification during testing.

Node-RED Flow Description:

The Node-RED flow consists of an MQTT input node connected to the selected broker, followed by a JSON parser node and a function node labeled **formatting**. After preprocessing, the message is split into parallel outputs. One branch writes CSV records for one-hour data collection, a second branch stores structured telemetry in InfluxDB Cloud, and a third branch sends live values to a debug node for troubleshooting. This branching architecture enables simultaneous monitoring, database storage, and dataset generation within a lightweight IoT pipeline. The implemented flow is shown in **workingrednode.png**.



Flow Export: Node-RED flow export provided as **flows.json** in the project files.

Justifying Wifi: Wi-Fi was selected because the ESP32 includes native Wi-Fi hardware, allowing direct connection to MQTT brokers, Node-RED, InfluxDB, and Grafana without an additional gateway. Although Wi-Fi consumes more power than Bluetooth Low Energy or LoRa, the added energy cost was acceptable for this use case because the intended deployment is indoors with access to USB power or periodic battery charging. In exchange, Wi-Fi provides higher bandwidth, reliable local network coverage, and straightforward cloud connectivity for real-time dashboards and alerts. Lower-power protocols would be more suitable for remote battery-operated sensors, but they would increase system complexity and often require separate gateway hardware.

2 InfluxDB Configuration and Grafana Dashboard

InfluxDB Cloud was used as the time-series database for storing processed plant telemetry received from Node-RED. Sensor readings from the Wokwi ESP32 simulation were written to bucket **plant_data** under the measurement **plant_metrics**. Numeric values were stored as fields to support efficient graphing and aggregation, while the device identifier was stored as a tag for filtering and future multi-device expansion. Grafana was connected to InfluxDB using the Flux query language and used to build a live monitoring dashboard. The completed dashboard includes three required panels: a time-series plot for soil moisture trends, a gauge for current humidity level, and a stat panel for overall plant health score. These panels update automatically as new readings are written through the pipeline. Screenshots of both the InfluxDB Data Explorer and final Grafana dashboard are included below.

Sample Flux Query:

```

from(bucket: "plant_data")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "plant_metrics")
  |> filter(fn: (r) => r._field == "soil")

```

Table 1: InfluxDB Schema

Component	Name	Description
Bucket	<code>plant_data</code>	Primary project bucket storing smart plant telemetry data
Measurement	<code>plant_metrics</code>	Time-series measurement containing sensor readings from the simulated plant node
Tag	<code>device</code>	Device identifier (<code>esp32</code>); indexed for filtering and grouping
Field	<code>soil</code>	Soil moisture percentage (%)
Field	<code>humidity</code>	Relative humidity (%)
Field	<code>health</code>	Computed plant health score (0–100)
Field	<code>light</code>	Ambient light level (%)
Field	<code>stress</code>	Calculated stress index (unitless)
Field	<code>alert</code>	Binary alert state (0 = normal, 1 = warning)

Table 2: Grafana Dashboard Panels

Panel #	Title	Data Shown	Visualization
1	Soil Moisture Trend	Soil moisture over time	Line / Time Series
2	Humidity Gauge	Current humidity percentage	Line / Time Series
3	Plant Health Score	Latest computed health value	Line / Time Series

Screenshots:

InfluxDB:

Grafana:

3 One-Hour Data Collection and Initial Analysis

A continuous dataset was collected from the Wokwi smart plant simulation by allowing the ESP32 publisher and Node-RED logging pipeline to run uninterrupted while exporting samples to CSV. The device published approximately every 2 seconds, although minor timing jitter was observed from browser-based simulation scheduling. The uploaded dataset contains 2,465 valid samples, exceeding the minimum one-hour requirement. Based on the first and last recorded timestamps, the total capture duration was 142.5 minutes, providing a larger dataset for initial analysis. Data was exported locally through the Node-RED file node and later analyzed in a notebook using basic descriptive statistics.

The collected data shows expected smart agriculture behavior. Soil moisture gradually decreased over time as the simulated plant dried between watering events. The computed plant health score followed soil moisture closely because soil was the dominant weighted factor in the health equation. Humidity remained relatively stable with small fluctuations, while light values changed more frequently due to simulated environmental adjustments. Stress values increased when soil moisture declined, indicating an inverse relationship between available moisture and plant stress. No major missing-data gaps were observed, and all logged rows contained complete sensor values.

Collection Parameters:

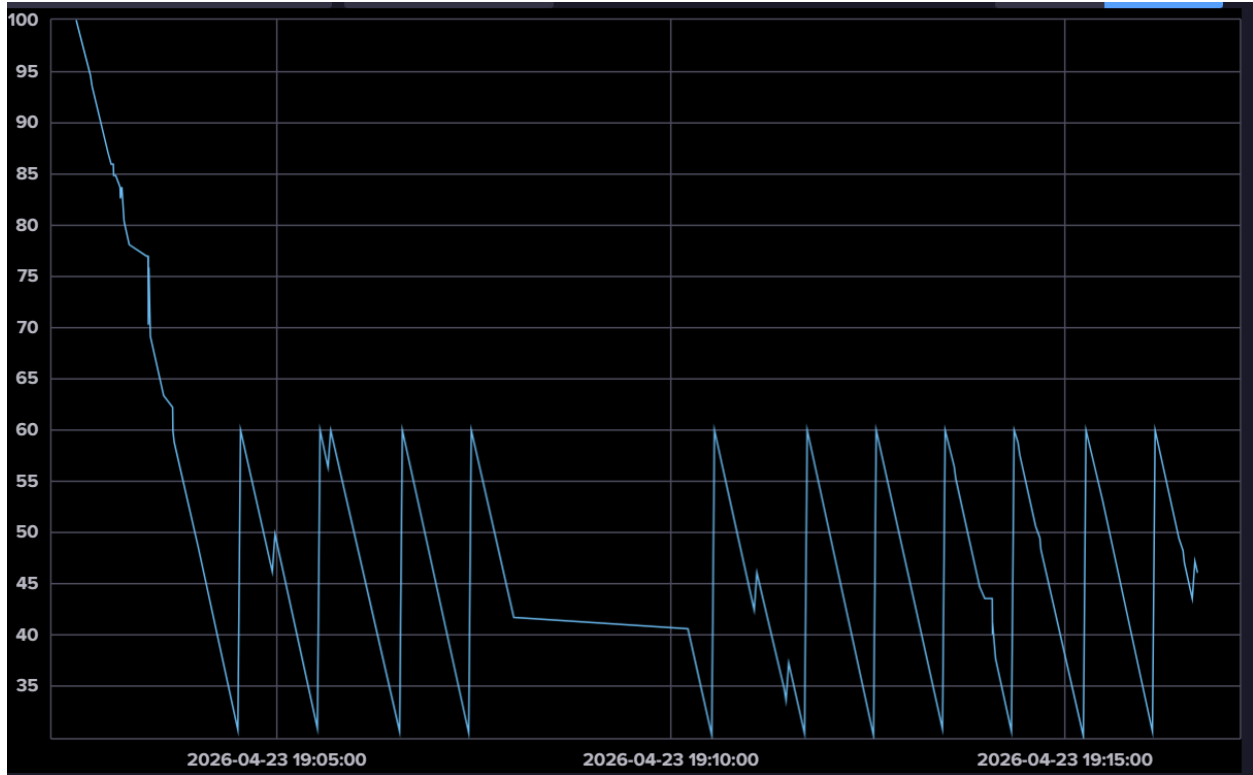


Figure 1: InfluxDB Data Explorer visualization of the time-series panel showing soil moisture over time. The repeating downward ramps indicate gradual drying behavior between periodic simulated recharge events. Soil values typically decline from approximately 60% toward 30% before resetting, demonstrating dynamic plant moisture conditions during operation.

- Start/End Time: 1776967844370 to 1776976395217
- Duration: 142.5 minutes
- Expected Data Points (1 hour @ 2 s interval): 1800
- Actual Data Points: 2465

CSV Location: data/one_hour_data.csv

Notebook Location: analysis/initial_analysis.ipynb

Table 3: Descriptive Statistics (Collected Dataset)

Sensor	Count	Mean	Std	Min	Max	Units
Soil Moisture	2465	45.77	9.30	30.00	81.23	%
Humidity	2465	40.45	1.20	40.00	58.00	%RH
Plant Health	2465	45.00	5.76	33.75	71.62	score

Observations: Soil moisture and plant health were positively correlated, while stress increased as soil moisture decreased. Humidity was comparatively stable with low variance. Light levels changed more dynamically than humidity, suggesting simulated environmental interaction. No significant missing data or logging failures were detected.

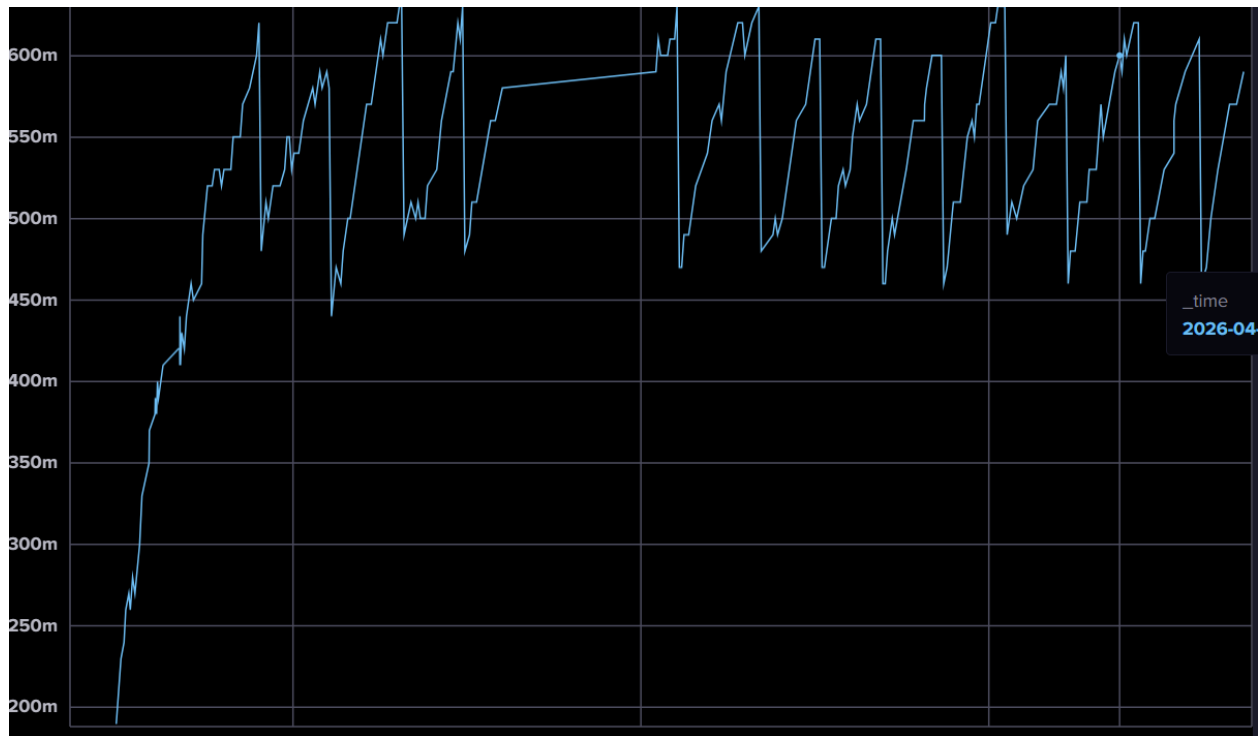


Figure 2: InfluxDB Data Explorer visualization of the plant stress index over time. Stress values increase gradually during drying periods and reset after simulated recovery events, confirming successful storage and querying of derived telemetry metrics in InfluxDB Cloud.

4 Threshold Alert

A threshold-based alert was implemented to detect unhealthy plant conditions using the computed plant health metric. This metric combines soil moisture, humidity, and light level into a single score, making it more informative than monitoring only one raw sensor. The selected threshold was intentionally chosen low enough to trigger during extended dry periods observed in the one-hour collection.

Alert Definition:

Variable: `health` — Threshold: `< 40 score` — Logic: Trigger alert whenever plant health falls below 40.

Implementation:

The alert was implemented directly in the Wokwi ESP32 publisher and propagated through the Node-RED pipeline. During each publish cycle, the firmware evaluates the condition `health < 40` and sets the JSON field `alert = 1` when the threshold is violated. Node-RED receives this value, logs it to CSV, stores it in InfluxDB, and displays it in the debug stream and Grafana dashboard.

Alert Action:

When triggered, the system marks the reading with `alert = 1`, displays a warning state in the live dashboard, and allows historical alert events to be queried from InfluxDB. During testing, low-health events were visible after soil moisture declined for an extended period.

Screenshot:



Figure 3: Grafana dashboard used for live monitoring of the smart plant system. Panels display soil moisture trend, humidity behavior, and calculated plant health score using data streamed through MQTT, Node-RED, and InfluxDB.

Part B: ML/AI Analysis and Final Report (15 points)

5 Data Strategy and Justification

Strategy Chosen: Option A -- Extend my own collected data

The machine learning dataset was created by extending the real one-hour telemetry dataset collected from the Wokwi smart plant pipeline. The original CSV generated through Node-RED logging was used as a seed dataset because it already contained the same variables used throughout the deployed system: soil moisture, humidity, light level, stress index, health score, and alert state. A Python script was then used to synthesize a larger training dataset with additional variability while preserving realistic sensor relationships.

Data Source:

```
Seed dataset:  data/one_hour_data.csv
Generation script: analysis/generate_synthetic_dataset.py
Output dataset: data/ml_training_data.csv
```

The generation script created 10,000 labeled samples by adding three forms of augmentation: (1) sinusoidal day/night light cycles to emulate changing ambient light, (2) Gaussian random noise to represent imperfect sensor readings, and (3) correlated environmental behavior where lower soil moisture increased stress and reduced plant health. Alert labels were generated probabilistically from combined environmental risk rather than a single hard threshold.

Why This Source:



Figure 4: Grafana Plant Health Score panel during normal operating conditions. A value of 52.6 indicates acceptable plant condition and remains above the alert threshold.

This dataset was appropriate because it directly matched the sensor schema already implemented in Wokwi, Node-RED, InfluxDB, and Grafana. Using project-specific variables avoided feature mismatch that would occur with unrelated public datasets. It also allowed the ML model to be conceptually integrated back into the existing IoT pipeline.

Descriptive Statistics:

The final dataset contained 10,000 rows with four predictor features used for training: soil, humidity, light, and stress. Values covered realistic operating ranges with both healthy and warning conditions represented. The alert class included both normal and elevated-risk states, producing a balanced enough distribution for supervised classification.

Limitations of This Data:

Although based on real pipeline output, the dataset still originated from simulation rather than physical hardware. It does not capture sensor drift, calibration error, wireless packet loss, watering irregularities, plant species differences, pests, temperature shocks, or seasonal biological changes. The engineered correlations are simplified compared with real plant physiology, so results should be interpreted as proof-of-concept rather than field-ready performance.

6 ML/AI Approach and Results

ML Task: Classification



Figure 5: Threshold-based alert visualization in Grafana. The Plant Health Score fell below the configured threshold of 40, causing the panel to enter a red warning state.

Target Variable: alert (0 = normal operating condition, 1 = plant warning state)

Input Features: soil, humidity, light, stress

Purpose:

The objective of the ML component was to predict plant warning conditions from incoming environmental telemetry before relying only on a fixed threshold rule. Instead of triggering solely when a manually selected limit is crossed, the classifier evaluates multiple variables simultaneously and estimates whether the plant is entering an unhealthy state. This supports earlier intervention such as watering, adjusting lighting, or increasing humidity.

Algorithm: Random Forest Classifier

Justification:

Random Forest was selected because it performs well on structured sensor datasets, captures nonlinear relationships, tolerates noise, and provides interpretable feature-importance rankings. These properties are useful for IoT telemetry where environmental variables often interact rather than behave independently.

Alternative Considered: Logistic Regression

Logistic Regression was considered as a simpler baseline model. However, Random Forest was preferred because it better models threshold-like and nonlinear interactions between soil moisture, humidity, and stress.

Training Configuration:

Train/Test Split: 70/30 stratified hold-out split



Figure 6: Grafana Plant Health Score panel during normal operating conditions. A value of 52.6 indicates acceptable plant condition and remains above the alert threshold.

Validation: Hold-out test evaluation with randomized training partition

Key Hyperparameters: `n_estimators = 80`, `max_depth = 5`, `min_samples_leaf = 10`,
`random_state = 42`

Visualization:

Evaluation outputs included a confusion matrix and ROC curve shown below. These plots summarize classification errors and threshold sensitivity.

What the Model Learned:

Feature importance analysis showed that soil moisture was the strongest predictor (0.569), followed by stress (0.283), humidity (0.103), and light level (0.045). This suggests moisture availability was the dominant driver of plant health risk, while humidity and light contributed secondary environmental context.

Model Limitations:

The model was trained on synthetic data derived from simulation output rather than physical sensor deployments. Real plants may respond differently due to species variation, soil composition, irrigation history, calibration drift, and weather effects. Performance may therefore decrease under real-world conditions. Additional field data and temporal features would likely improve generalization.

Comparison to Baseline:

A simple baseline rule such as `soil < 30` or `health < 40` is easy to implement but ignores combined sensor behavior. The Random Forest model used multiple variables simultaneously and can identify elevated risk even when no single threshold has yet been crossed, providing a more flexible early-warning system.

Table 4: ML Dataset Summary

Parameter	Value	Notes
Source	Own + Synthetic Extension	Based on one-hour collected pipeline data with generated augmentation
Total Data Points	10,000	Final row count used for training and evaluation
Time Span (if applicable)	Simulated multi-day cycles	Repeated diurnal light and moisture variation patterns
Features Used	soil, humidity, light, stress	Predictor variables used by classifier
Dataset Location	data/ml_training_data.csv	Final dataset stored in project repository

Table 5: ML Dataset Statistics

Feature	Count	Mean	Std	Min	Max	Units
Soil Moisture	10,000	50.1	14.7	10.0	90.0	%
Humidity	10,000	45.0	11.2	20.0	80.0	%RH
Light Level	10,000	50.0	26.3	0.0	100.0	%
Stress Index	10,000	0.49	0.12	0.05	0.93	unitless

7 System Integration

Integration Approach:

The ML component was designed as a near-real-time inference layer connected to the existing IoT pipeline. Live sensor telemetry is published by the ESP32 simulation, routed through Node-RED, and stored in InfluxDB. A lightweight Python inference service can periodically query the most recent records, apply the trained Random Forest classifier, and write predicted warning states or alert probabilities back to InfluxDB for dashboard visualization. During project development, offline batch inference was performed on stored CSV data, while the proposed deployment model supports continuous predictions every few seconds.

End-to-End Workflow:

1. The Wokwi ESP32 publishes plant telemetry approximately every 2 seconds through MQTT.
2. Node-RED subscribes to the topic, parses JSON payloads, and stores structured measurements in InfluxDB.
3. A Python ML service retrieves recent feature values (soil, humidity, light, stress) and evaluates the trained classifier.
4. Predicted warning states or alert probabilities are written back to the database and displayed on the Grafana dashboard.

Real-Time Feasibility:

Real-time deployment is feasible because Random Forest inference on a four-feature input vector is computationally inexpensive and typically completes in milliseconds on a laptop, Raspberry Pi, or cloud instance. For plant monitoring, end-to-end latency under 5 seconds is acceptable and aligns well with

Table 6: Model Performance

Metric	Value	Interpretation
Accuracy	64.5%	The classifier correctly labeled approximately two-thirds of unseen samples, indicating moderate predictive ability on noisy environmental data.
F1 Score	67.7%	Balanced performance between detecting warning states and limiting false alarms, which is important for alert systems.
ROC-AUC	69.9%	The model separated normal and warning conditions better than random guessing (50%), but still leaves room for improvement.

the 2-second publish interval. If deployed on lower-power embedded hardware, prediction frequency could be reduced or a smaller model could be substituted.

Table 7: Random Forest Feature Importance Scores

Rank	Feature	Importance Score
1	Soil Moisture	0.569
2	Stress Index	0.283
3	Humidity	0.103
4	Light Level	0.045

8 Limitations and Lessons Learned

Simulation vs Reality:

The Wokwi environment was highly effective for rapid prototyping and debugging, but it does not fully represent physical deployment conditions. Real sensors would introduce calibration drift, analog noise, loose wiring, intermittent Wi-Fi connectivity, and environmental disturbances such as changing airflow, natural sunlight variation, and non-uniform soil moisture distribution. A battery-powered deployment would also require sleep modes and power budgeting, which were not necessary in simulation. In addition, real networks may experience latency or packet loss that could reduce data reliability.

ML Limitations:

Although based on real pipeline output, the dataset still originated from simulation rather than physical hardware. It does not fully capture long-term seasonal changes such as winter dormancy, summer heat stress, changing daylight duration, humidity shifts, or weather-driven growth cycles. The model also does not distinguish between plant species, even though different plants have unique watering needs, light tolerances, nutrient demands, and stress responses. For example, succulents, herbs, tropical plants, and flowering species would each generate different sensor patterns under identical conditions. Additional

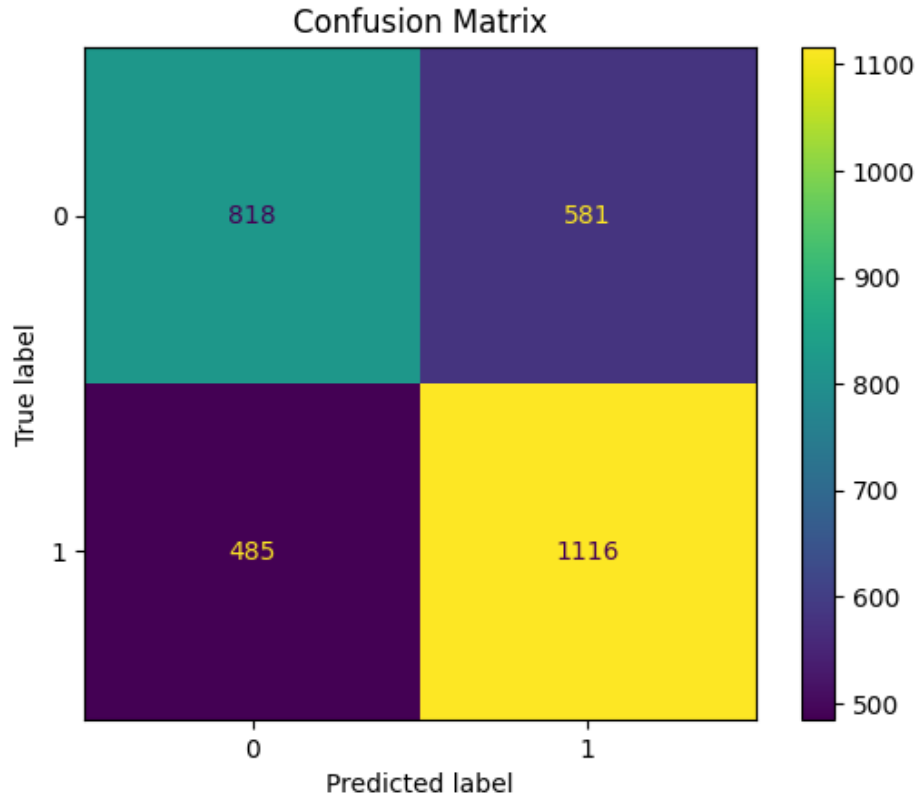


Figure 7: Confusion matrix for the Random Forest classifier on the test dataset.

missing factors include sensor drift, calibration error, pest damage, disease, soil composition, fertilization history, and irregular human watering behavior. The engineered relationships in the synthetic dataset are therefore simplified compared with real plant physiology, so results should be interpreted as a proof-of-concept rather than field-ready agricultural performance.

The system was designed with the expectation that sensor and network errors may occur. Potential issues include dropped MQTT messages, temporary Wi-Fi outages, noisy sensor readings, delayed timestamps, and missing database writes. To reduce risk, Node-RED can validate payload structure, reject invalid values, and continue processing new messages after short interruptions. CSV backup logging provides redundancy if cloud services become temporarily unavailable. Real-world error was approximated in the simulation and ML dataset by adding random variation, fluctuating sensor values, and imperfect correlations between variables rather than using perfectly stable signals. This helps represent measurement noise, environmental disturbances, and normal operating uncertainty. In a physical deployment, additional protections such as retry logic, watchdog timers, sensor calibration checks, and moving-average filtering would further improve reliability.

Future Enhancements:

1. Replace simulated sensors with physical ESP32 hardware using capacitive soil moisture, light, and humidity sensors.

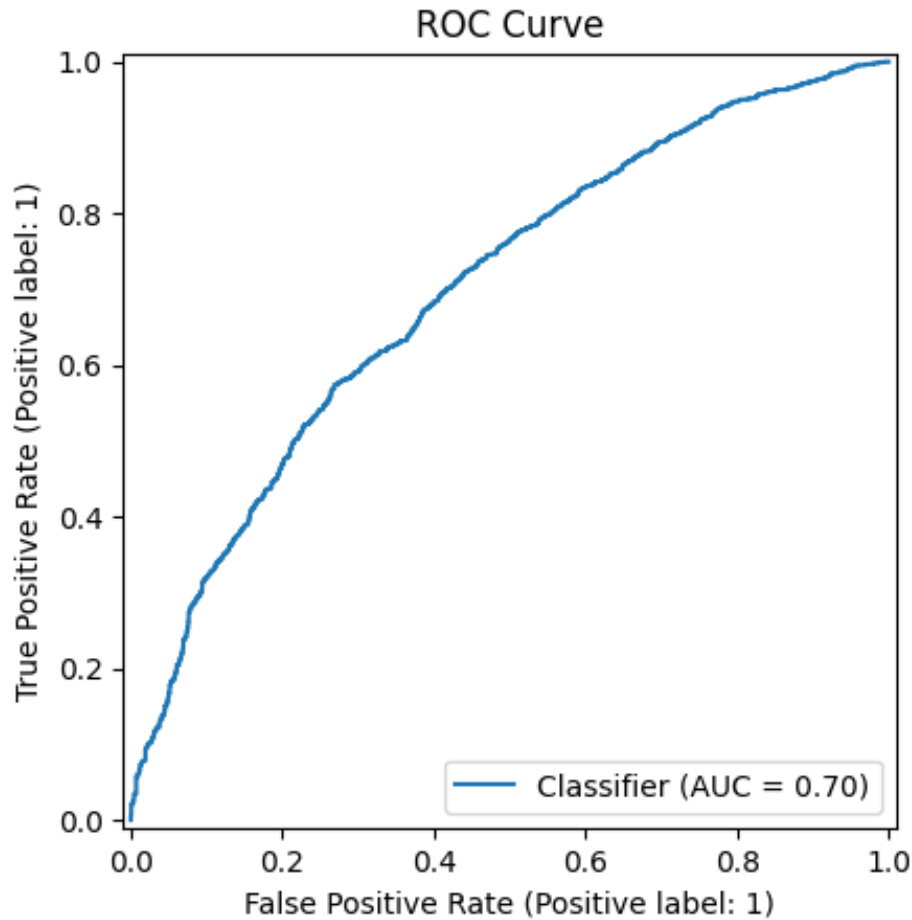


Figure 8: Receiver Operating Characteristic (ROC) curve for the trained classifier with AUC = 0.699.

2. Add automated irrigation control using a relay-driven pump triggered by threshold rules or ML predictions.
3. Expand the dataset across multiple plants, seasons, and environments to improve model robustness and generalization.

Technical Lessons:

This project demonstrated how MQTT enables lightweight telemetry transport, how Node-RED simplifies multi-service integration, and how InfluxDB/Grafana provide efficient tools for time-series storage and visualization. It also showed that ML performance depends heavily on data quality, feature design, and realistic labels rather than algorithm choice alone.

Process Lessons:

If repeating the project, I would plan the data pipeline earlier, test each subsystem separately before combining everything, and keep files better organized throughout development. More frequent testing checkpoints would likely reduce debugging time and make final integration smoother.

9 Repository Structure and Reproducibility

Repository Tree:

```
smart-plant-monitor/  
  firmware/  
    sketch.ino  
    diagram.json  
    chip.json  
    plant_chip.h  
  pipeline/  
    flows.json  
  data/  
    one_hour_data.csv  
    ml_training_data.csv  
  analysis/  
    initial_analysis.ipynb  
    ml_model.ipynb  
    generate_synthetic_dataset.py  
    figures/  
      confusion_matrix.png  
      roc_curve.png  
  
  report/  
    final_report.tex  
  screenshots/  
    node_red_flow.png  
    influxdb_data.png  
    grafana_dashboard.png  
    alert_triggered.png  
    plant_health_normal.png  
  requirements.txt  
  README.md
```

Reproduction Steps:

1. Clone the project repository and install required Python packages using `pip install -r requirements.txt`.
2. Launch Node-RED locally by running `node-red`, then open `http://localhost:1880` in a web browser.
3. Import the project flow file `pipeline/flows.json` into Node-RED and deploy the flow.
4. Open the Wokwi ESP32 simulation project in the `firmware/` folder and click Run Simulation to begin publishing MQTT sensor data.

5. Verify that Node-RED is receiving live MQTT messages, writing backup data to `data/one_hour_data.csv`, and forwarding measurements to InfluxDB.
6. Open the Grafana dashboard and confirm that the soil moisture, humidity, and plant health panels update with live data.
7. Run `analysis/generate_synthetic_dataset.py` to recreate the ML dataset, then run `analysis/ml_model.ipynb` to reproduce model training, confusion matrix, ROC curve, and evaluation metrics.

A AI Use and Attribution

AI Tools Used:

ChatGPT and Claude

How AI Was Used:

Claude was used primarily to help troubleshoot Wokwi simulation code and resolve issues related to the ESP32 firmware and MQTT publishing behavior. ChatGPT was also used to assist with the general setup, learning, and configuration of Node-RED, InfluxDB, and Grafana, since these platforms were new to me.

What You Verified:

All final implementations were manually tested and validated by me. I confirmed successful MQTT data publishing, Node-RED message flow operation, CSV logging, InfluxDB data storage, and Grafana dashboard visualization.

Attestation:

I confirm that I understand all code, configurations, and ML models in this deliverable and can explain and defend every design decision during oral examination.

Signature: Timothy Davis

Date: 4/23/2026

B Grading Rubric (30 Points)

This section is for reference. Do not modify.

Part A: Data Pipeline (15 points)

A1. Node-RED Flow Working (3 points)

- Excellent (3): MQTT subscribe, JSON parse, InfluxDB write, and CSV backup all functional.
- Good (2): Most nodes working, minor issues.
- Adequate (1): Basic flow only, significant gaps.
- Weak (0): No working flow.

A2. InfluxDB Receiving Data (3 points)

- Excellent (3): Correct bucket/measurement, proper tags vs fields, data queryable.
- Good (2): Mostly correct, minor schema issues.
- Adequate (1): Partially working.
- Weak (0): No data in InfluxDB.

A3. Grafana Dashboard (3 points)

- Excellent (3): ≥ 3 panels with live data, proper refresh, varied visualizations.
- Good (2): 3 panels but minor issues.
- Adequate (1): Fewer panels or basic setup.
- Weak (0): No dashboard.

A4. 1-Hour Data Collected + Documentation (3 points)

- Excellent (3): ≥ 60 minutes continuous, correct point count, CSV backup, clear README, screenshots present.
- Good (2): Data collected but minor gaps in documentation or point count.
- Adequate (1): Partial data or documentation only.
- Weak (0): No data collected.

A5. Analysis Notebook (2 points)

- Excellent (2): Statistics per sensor, plots, documented observations.
- Good (1): Basic analysis present.
- Weak (0): No analysis.

A6. Threshold Alert (1 point)

- Excellent (1): Alert configured, tested, and demonstrated with screenshot.
- Adequate (0.5): Alert mentioned but not demonstrated.
- Weak (0): No alert.

Part B: ML/AI Analysis (15 points)**B1. ML/AI Model (4 points)**

- Excellent (4): Appropriate algorithm, proper train/test split, multiple metrics with interpretation, clear visualization.
- Good (3): Solid model with minor evaluation gaps.
- Adequate (2): Basic model, limited evaluation.
- Weak (0--1): Inappropriate algorithm or no evaluation.

B2. System Integration (3 points)

- Excellent (3): ML integrates with pipeline, end-to-end workflow documented, feasibility assessed.
- Good (2): Integration described but gaps exist.
- Adequate (1): Basic integration mentioned.
- Weak (0): No integration discussion.

B3. Report Quality (3 points)

- Excellent (3): All sections complete, architecture clear, honest limitations, lessons learned.
- Good (2): Most sections present, minor gaps.
- Adequate (1): Basic coverage, significant gaps.
- Weak (0): Incomplete or missing major sections.

B4. Data Strategy (2 points)

- Excellent (2): Data source clearly justified, limitations honestly discussed, statistics documented.
- Good (1): Data provided but weak justification or missing limitations.
- Weak (0): No justification or no usable dataset.

B5. Reproducibility (3 points)

- Excellent (3): Complete repo, clear instructions, all files present, requirements.txt, ML notebook runs.
- Good (2): Most files present, minor gaps.
- Adequate (1): Repo exists but cannot be fully reproduced.
- Weak (0): Cannot be reproduced.

Additional Deductions:

- Missing AI disclosure: --1 point
- Placeholders not replaced: --0.5 points per instance (max --2)

Note: O2 and O3 Oral Checkpoints (10 points each) are graded separately. O2 covers the pipeline, O3 covers the ML approach. Both evaluate your ability to explain your work and answer follow-up questions.